Provenance Lab

A two-layer, contrastively trained framework for detecting AI-generated essays, code, images and speech

Sanatan Sharma (CO23355) Ryanveer Singh (CO23353)
Mentor: Prof. Varun Gupta CS756 Project-I

AI-made content is now hard to tell apart

Academic integrity

Essays and assignments written by chatbots

Code authenticity

AI-written code submitted as original work

Misinformation

Photorealistic synthetic images spread as real

Voice fraud

Cloned voices used in scam calls

One detector per problem doesn't scale. We need one framework that covers all four, works on generators it hasn't seen, and says why it decided.

What existing detectors get wrong

Single modality

Text detectors, image detectors and voice anti-spoofing are separate systems with separate interfaces.

Break on new generators

Classifiers learn one model's quirks and fail when a newer model arrives.

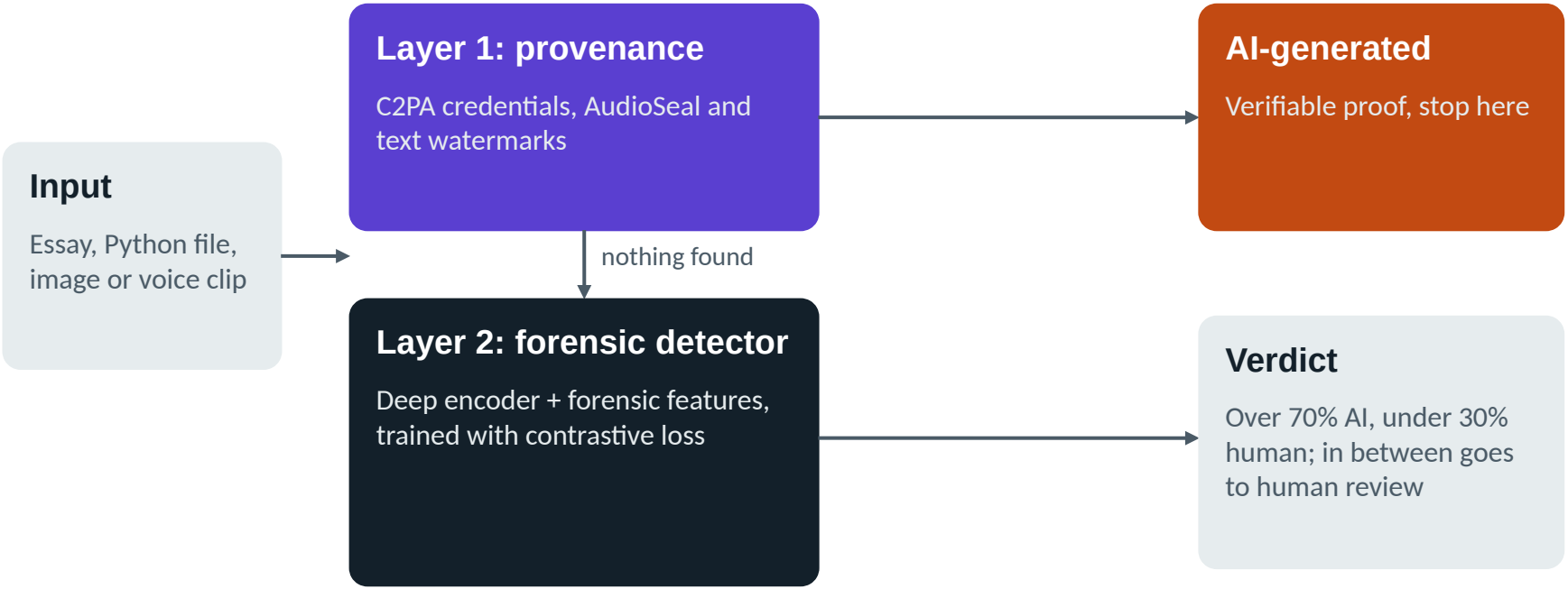
Watermarks only help if present

SynthID and C2PA prove origin, but most tools don't add them and edits strip them.

Over-confident verdicts

A forced yes/no wrongly accuses people; there is no 'not sure' outcome.

Two layers: proof first, then a trained detector



Layer 2: a specialised detector per content type

	Deep encoder	Hand-crafted forensic signals (fused with the encoder)
Essays	BERT (bert-base-uncased)	Perplexity and its spread under DistilGPT-2, sentence burstiness, vocabulary richness
Code	CodeBERT	Syntax-tree depth and node mix, function length, identifier style, comment ratio
Images	Vision Transformer + frequency CNN (224 px)	2-D FFT spectrum branch, radial power profile, noise-residual statistics
Voice	wav2vec 2.0 (frozen) + linear head	Compared with a spectrogram CNN using MFCC, spectral-shape and phase cues

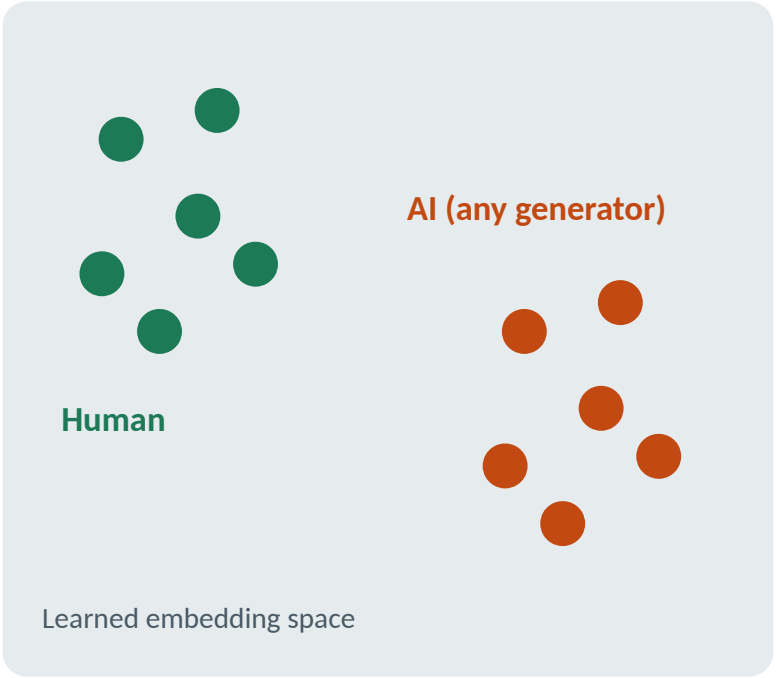
Contrastive learning: what all AI content shares

Every detector is trained with two losses on the fused embedding:

- Binary cross-entropy for the human/AI decision
- Supervised contrastive loss (SupCon, temperature 0.1) that pulls same-label samples together and pushes human and AI apart

L = BCE + 0.5 × SupCon

Why: the model is rewarded for a shared notion of 'AI-made' across all generators in a batch, not one generator's quirks, which should help on unseen models.



Real public benchmarks, leakage-free splits

Content	Dataset	Human vs AI	Train / test	Leakage control
Essays	HC3 + MAGE	Humans vs ChatGPT and 27 LLMs	16,000 / 1,000 + 1,000	By question; OPT held out
Code	CoDET-M4 (Python)	GitHub, LeetCode, Codeforces vs 5 LLMs	12,000 / 1,000	Official split
Images	GenImage + CIFAKE	Real photos vs 7 modern generators	14,000 / 3,500	Official split
Voice	ASVspoof 2019 + modern TTS	Bona fide vs 2019 attacks, ElevenLabs, Polly…	4,339 / 1,527	By speaker / recording

All subsets are class-balanced. 10% of each training set is held out for validation. These are deliberately small subsets so the full pipeline trains on one laptop; full-scale training is future work.

Training built to avoid over- and underfitting

Validation hold-out

10% of each training set, never trained on

Early stopping

Stop after 2 epochs without validation improvement; keep the best epoch

Regularisation

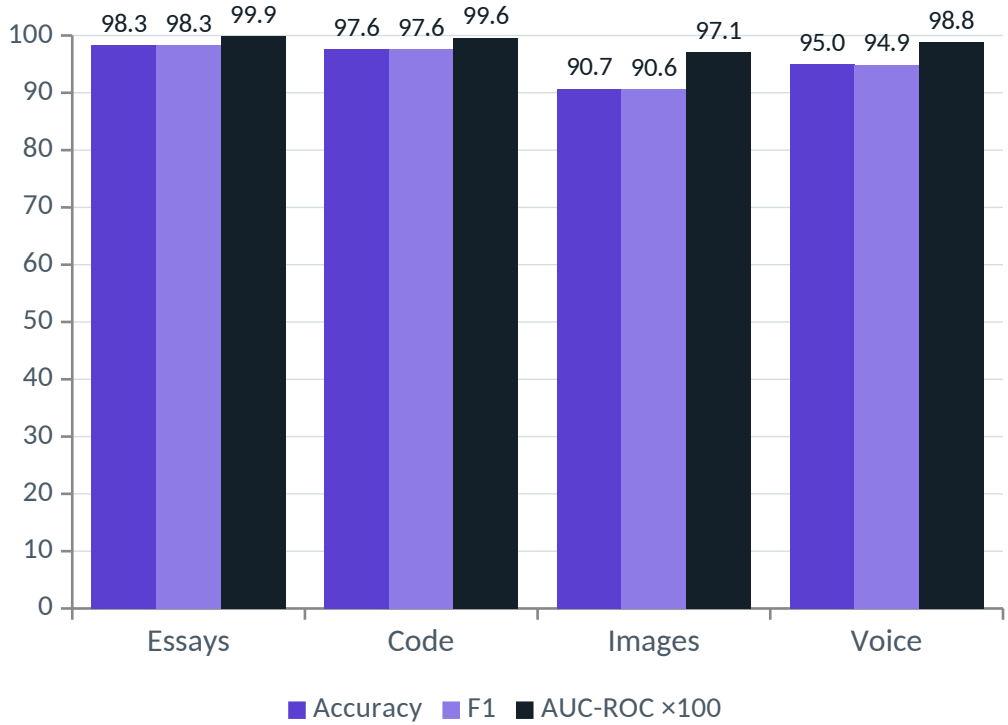
Dropout, weight decay, SpecAugment for voice, gradient checkpointing

Shortcut removal

HC3 spacing and \n escapes; ASVspoof silence; GenImage JPEG vs PNG

Hardware: Apple M4 laptop, 16 GB unified memory, PyTorch on the Metal GPU, every job capped at 3 GB by a memory watchdog.

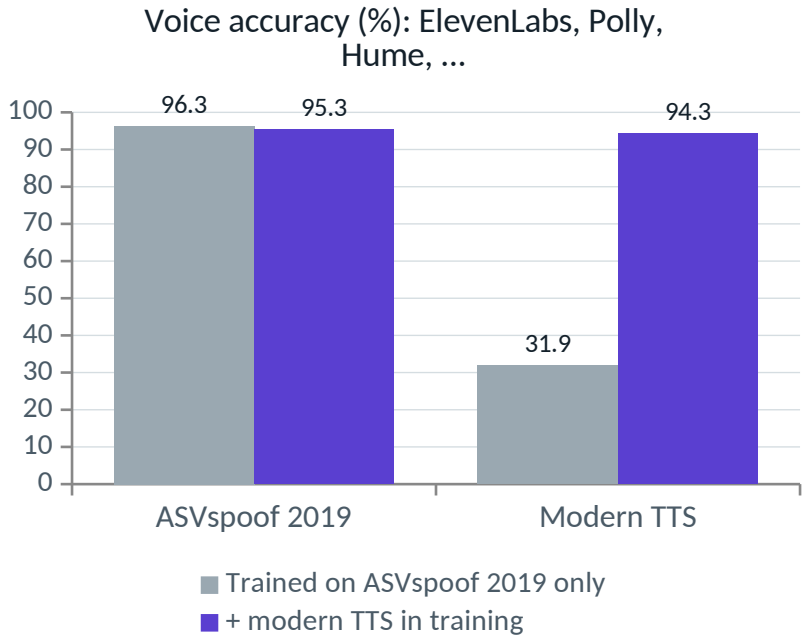
Results on held-out test data

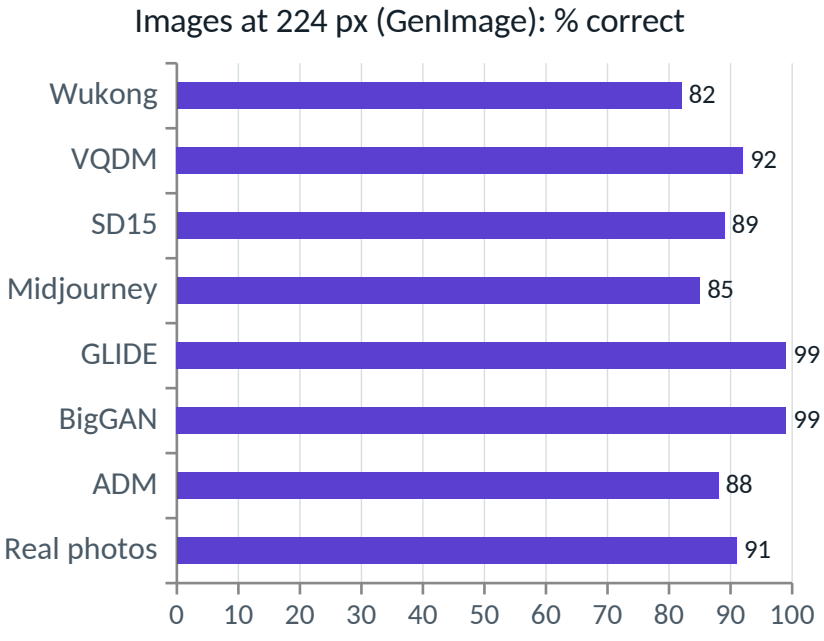


Content	Prec.	Recall	Test n
Essays	97.1%	99.6%	1000
Code	96.5%	98.8%	1000
Images	90.8%	90.5%	3500
Voice	96.2%	93.6%	1527

Baseline results on small subsets; full datasets and longer training are expected to improve them.

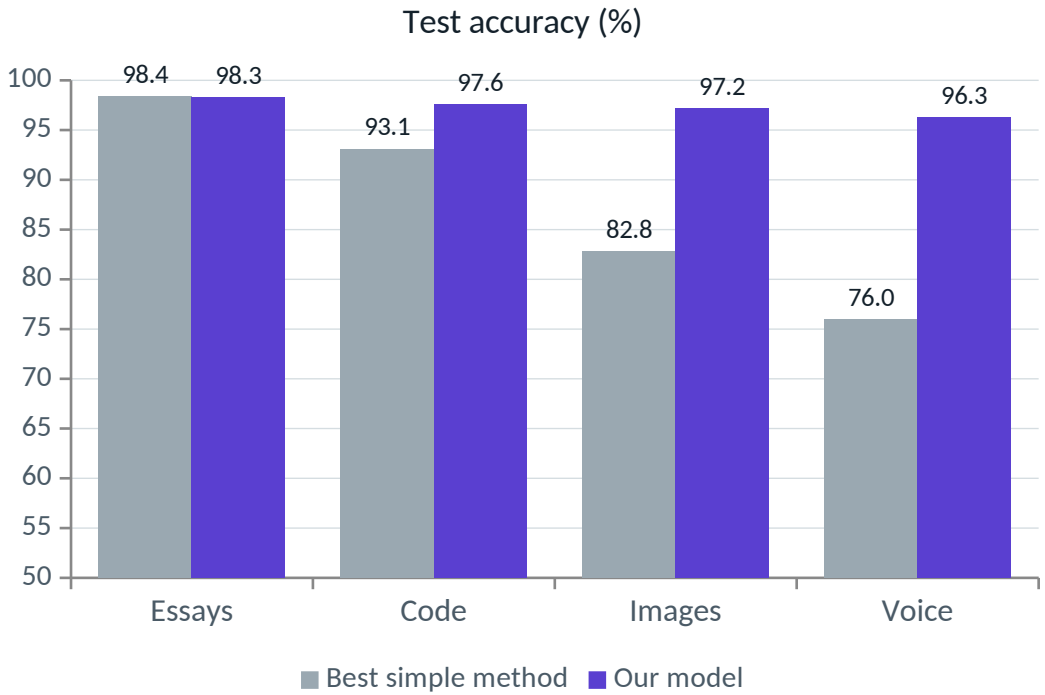
Built for today's generators, not 2019's





A 2019-trained voice model flags real speech and misses modern clones (31.9%); adding modern TTS fixes it. With Midjourney removed from training, 56.4% of Midjourney images are still caught (vs 84.8%).

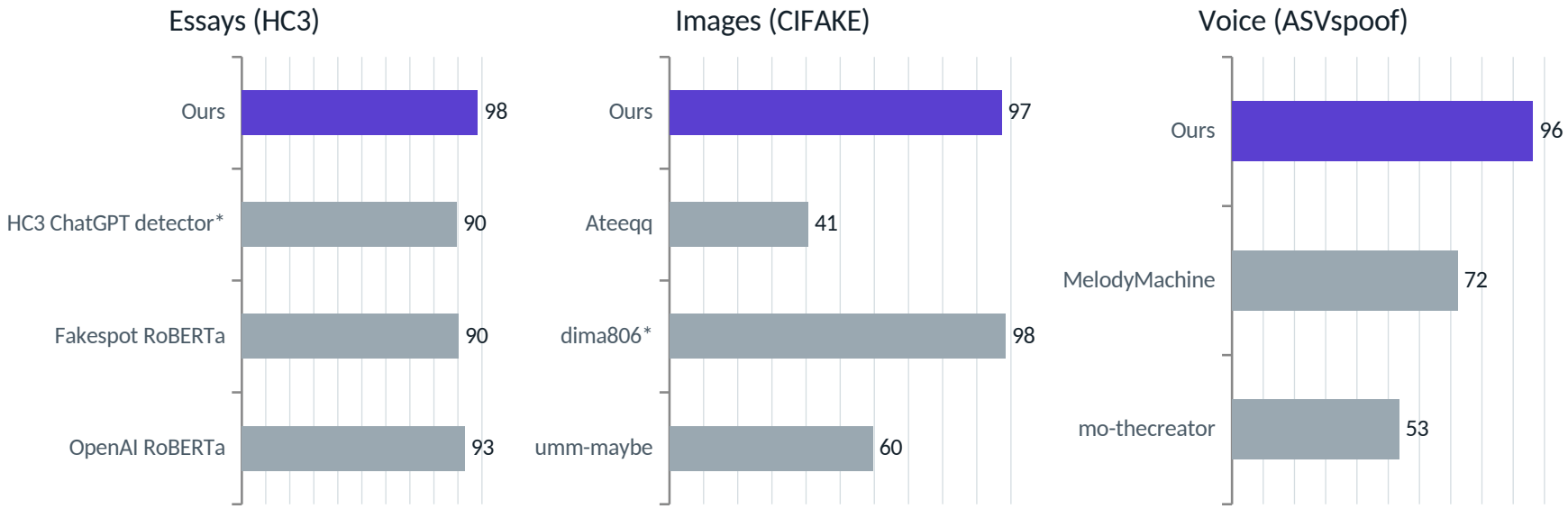
Beating simple methods on the same data



Simple methods: perplexity threshold, TF-IDF + logistic regression, forensic features + logistic regression

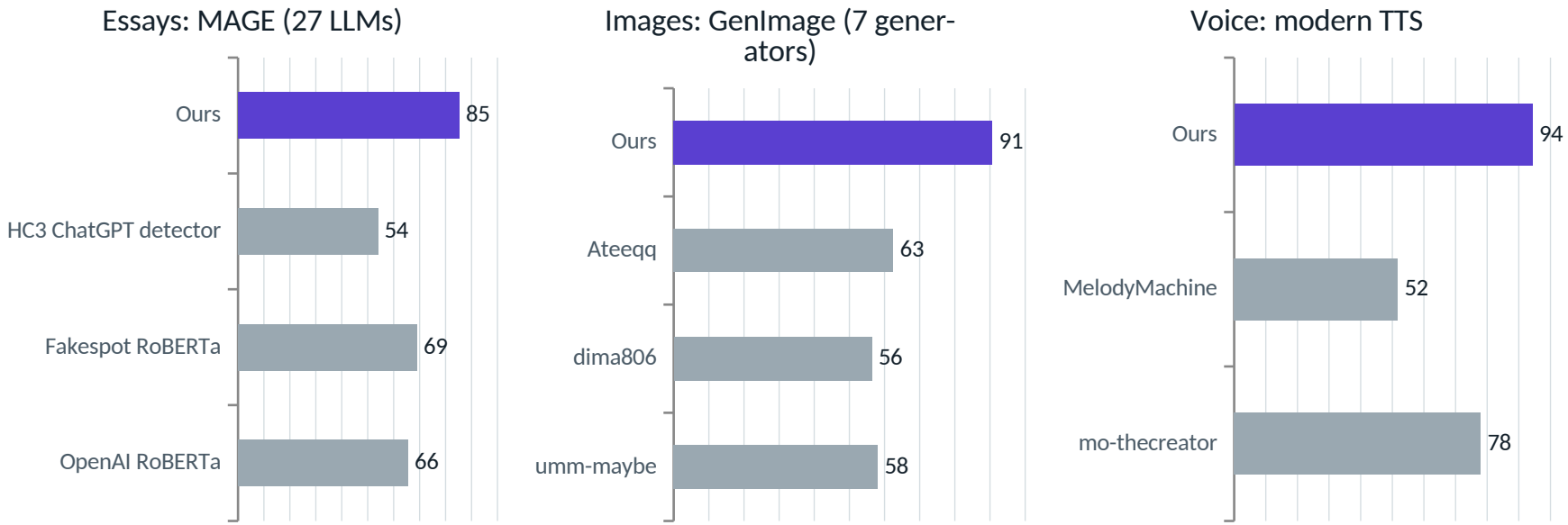
- **Essays: -0.1 pts (±1.1), within noise**
- **Code: +4.5 pts (±1.8)**
- **Images: +14.4 pts (±1.8)**
- **Voice: +20.3 pts (±2.9)**

Compared with detectors people actually use



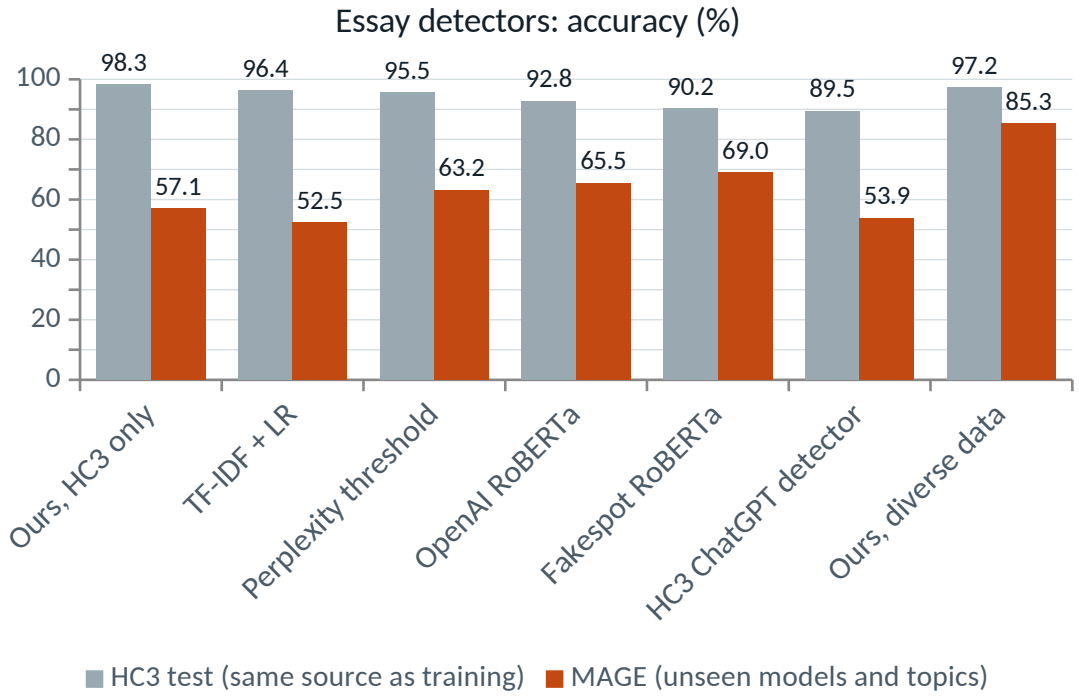
* trained on that same benchmark (home advantage). Commercial tools are closed: Weber-Wulff et al. (2023) found 14 tools incl. Turnitin "neither accurate nor reliable"; RAID (Dugan et al., ACL 2024) found detectors "easily fooled" by unseen models and simple attacks.

Today's AI content: ours vs public detectors



None of these test sets was used to train any public model, so this is the fair comparison. Every model is run unchanged; essays use our calibrated diverse-data model, images and voice our modern-data models.

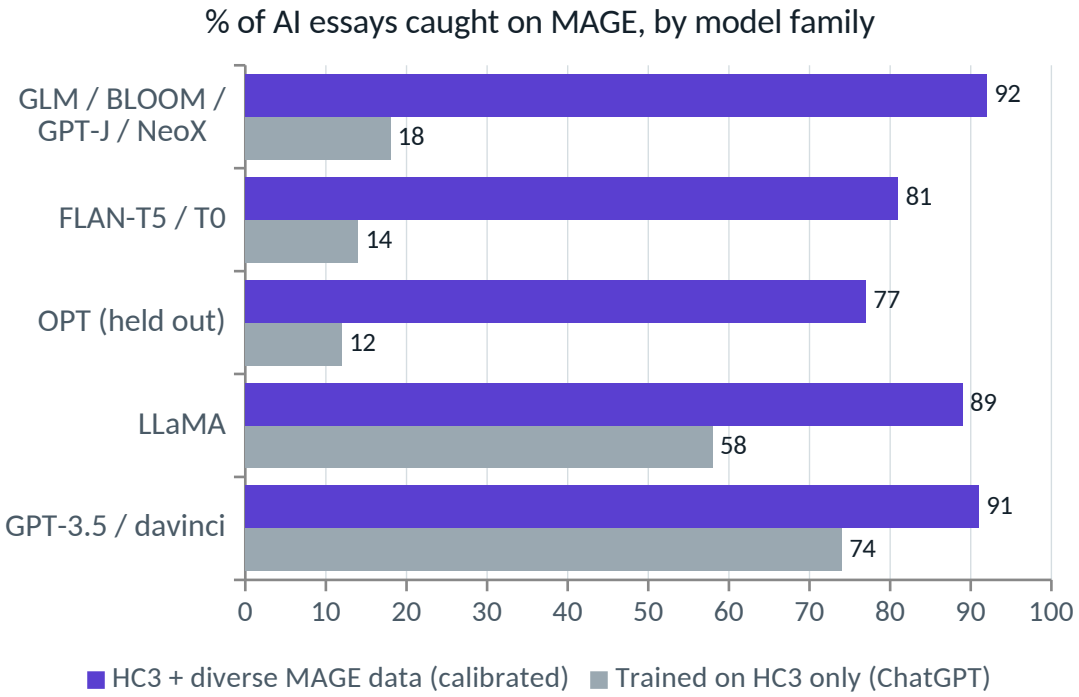
Honest check: no detector is 100% accurate



Tested without retraining on 1000 MAGE texts: 14 domains, 27 AI models never seen in training.

Every method drops. Benchmark accuracy overstates real-world accuracy, which is why we report this gap, keep an 'uncertain' band, and put verifiable provenance first.

The fix: train on many AI models, not one



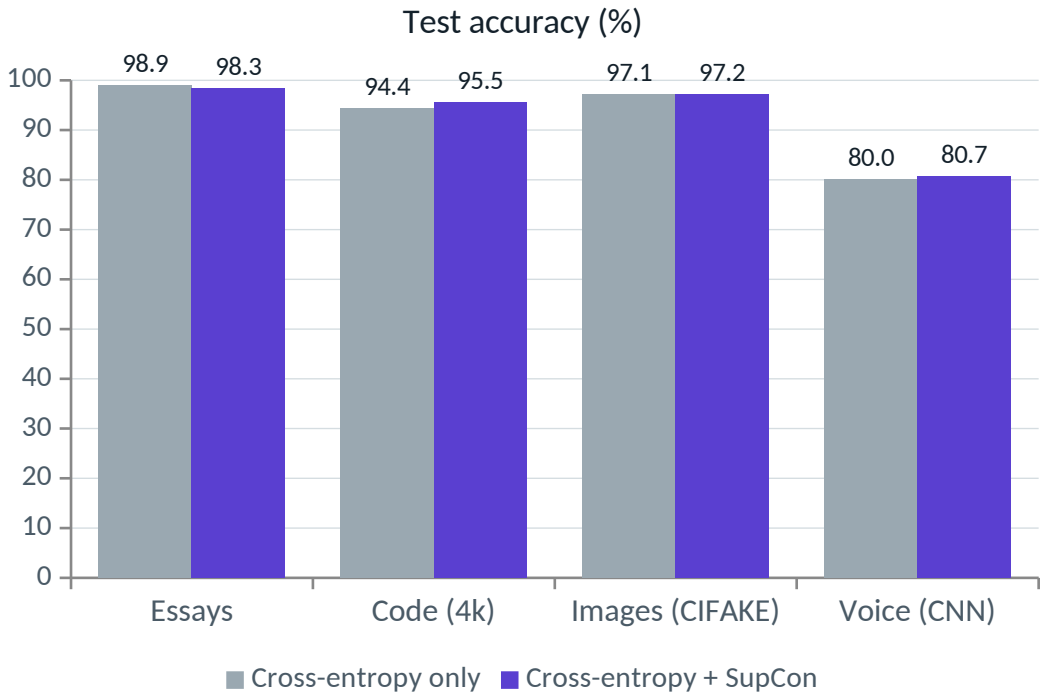
MAGE accuracy: 57.1% → 85.3%

HC3 accuracy stays 97.2%; 86.2% of human texts still cleared

Threshold calibrated on MAGE validation data (0.85), never on test.

OPT was never in training: its bar is a true unseen-generator result.

Does contrastive learning help?

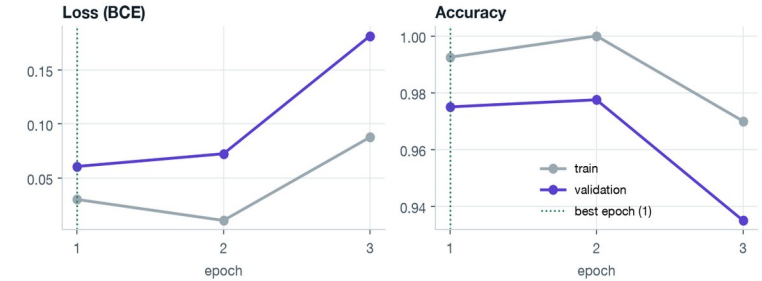


- Essays: -0.6 pts (±1.0), within noise
- Code (4k): +1.1 pts (±1.9), within noise
- Images (CIFAKE): +0.1 pts (±1.0), within noise
- Voice (CNN): +0.7 pts (±3.5), within noise

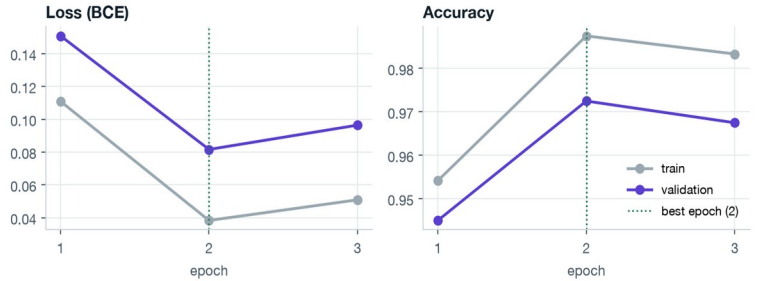
± is the 95% confidence interval of the difference.

It generalises: train and validation move together

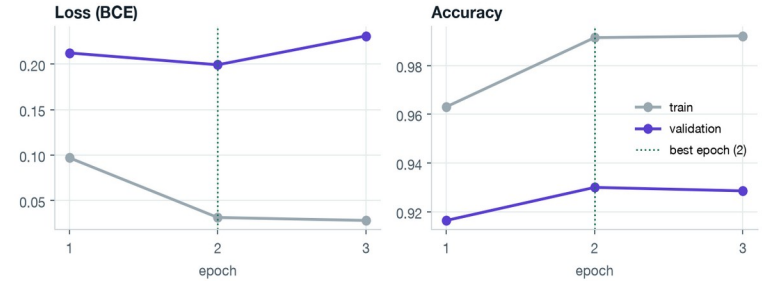
Essays



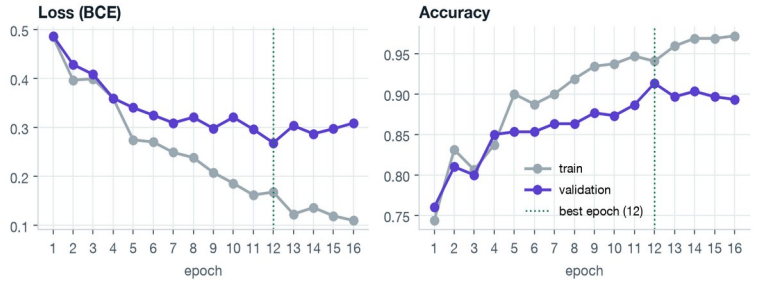
Code



Images



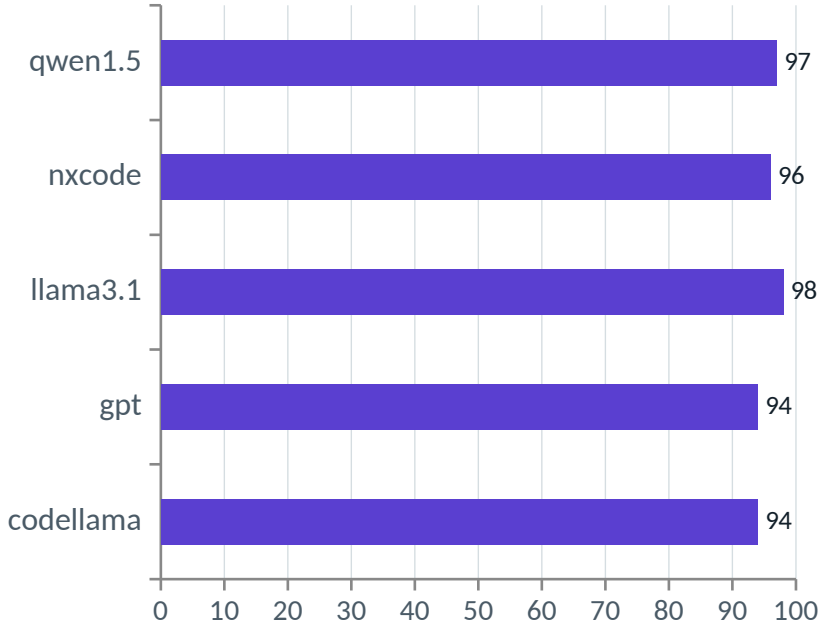
Voice (spectrogram CNN)



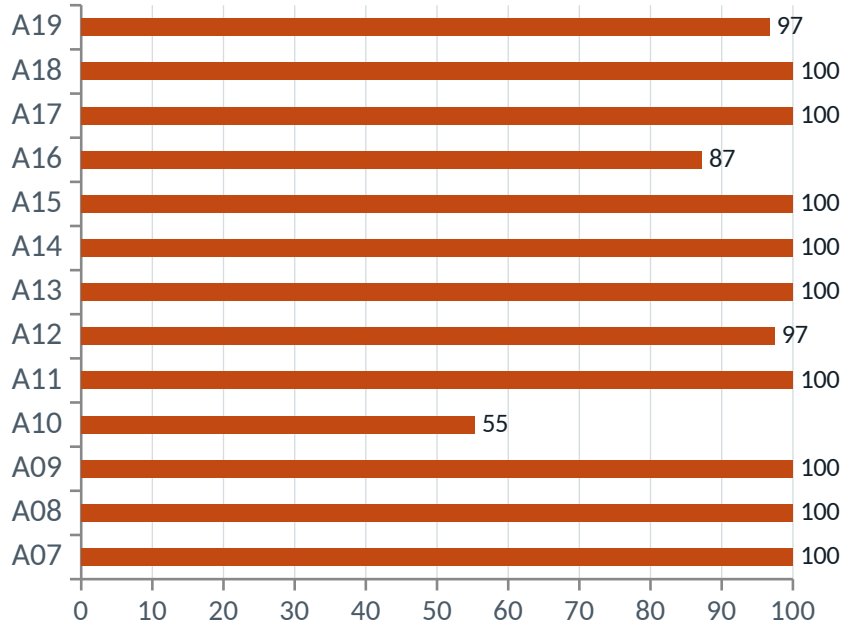
Train minus test accuracy (pts): Essays +1.0 Code +1.2 Images +8.5 Voice +4.2. Voice is tested on unseen speakers.

Which generators and attacks are hardest

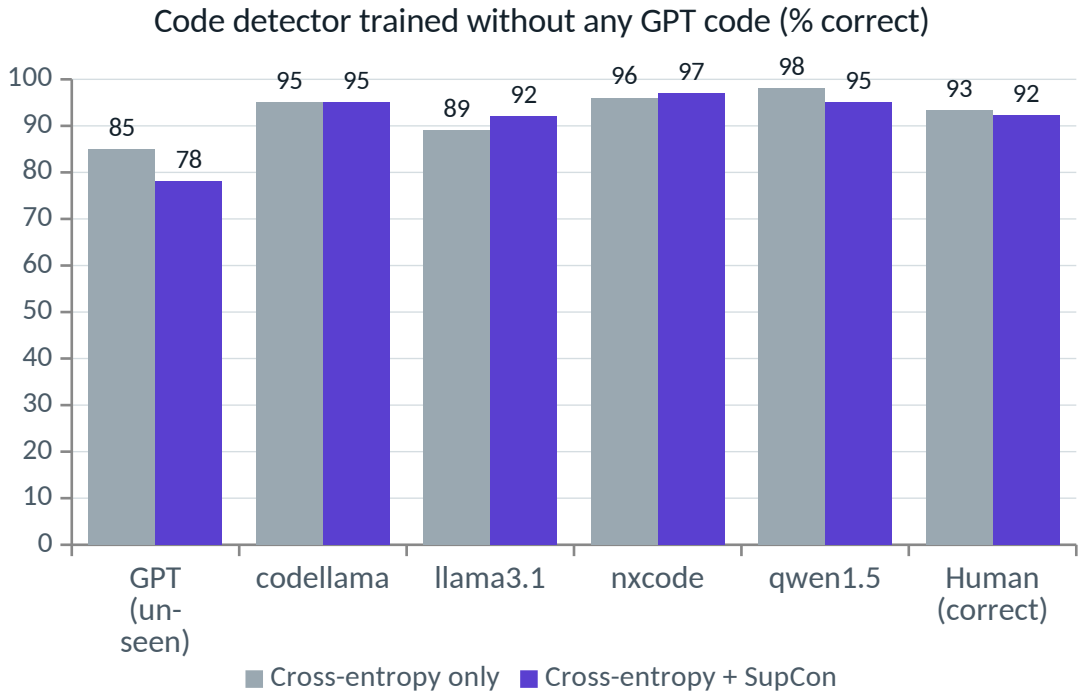
Code: % of AI samples caught, per LLM



Voice: % of spoofs caught, per attack



Catching a generator it never saw in training



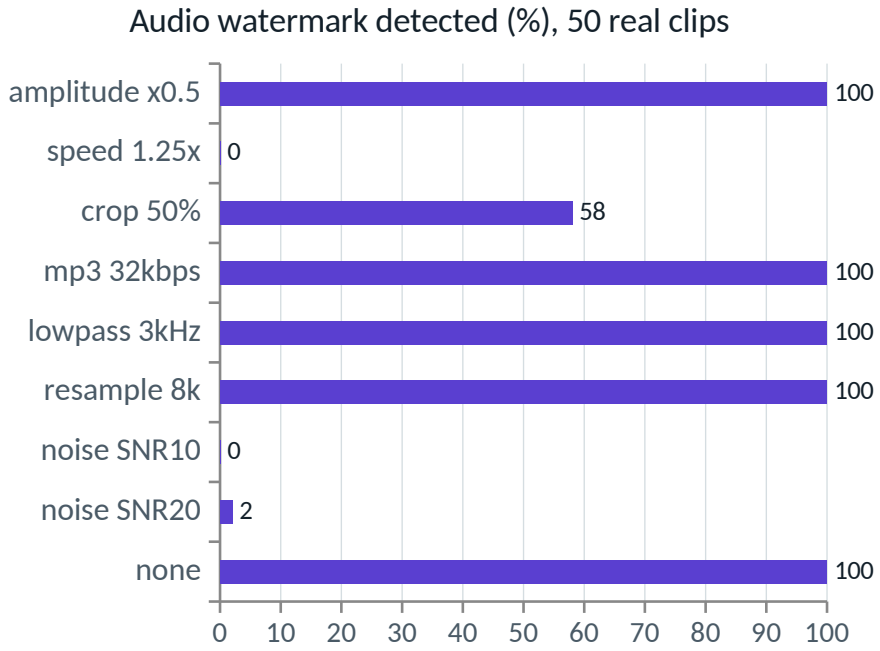
Leave-one-generator-out test

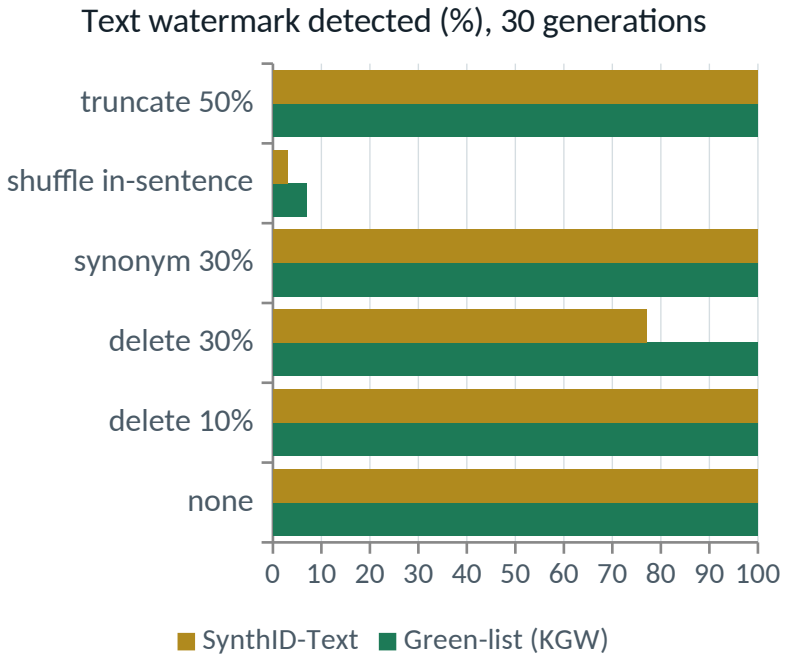
All GPT-written code removed from training; the test set still contains it.

Unseen GPT code is still caught 78–85% of the time (vs 94% when GPT is in training).

SupCon vs cross-entropy: -7.0 pts (±10.7), within noise. At this scale contrastive learning gave no measurable gain.

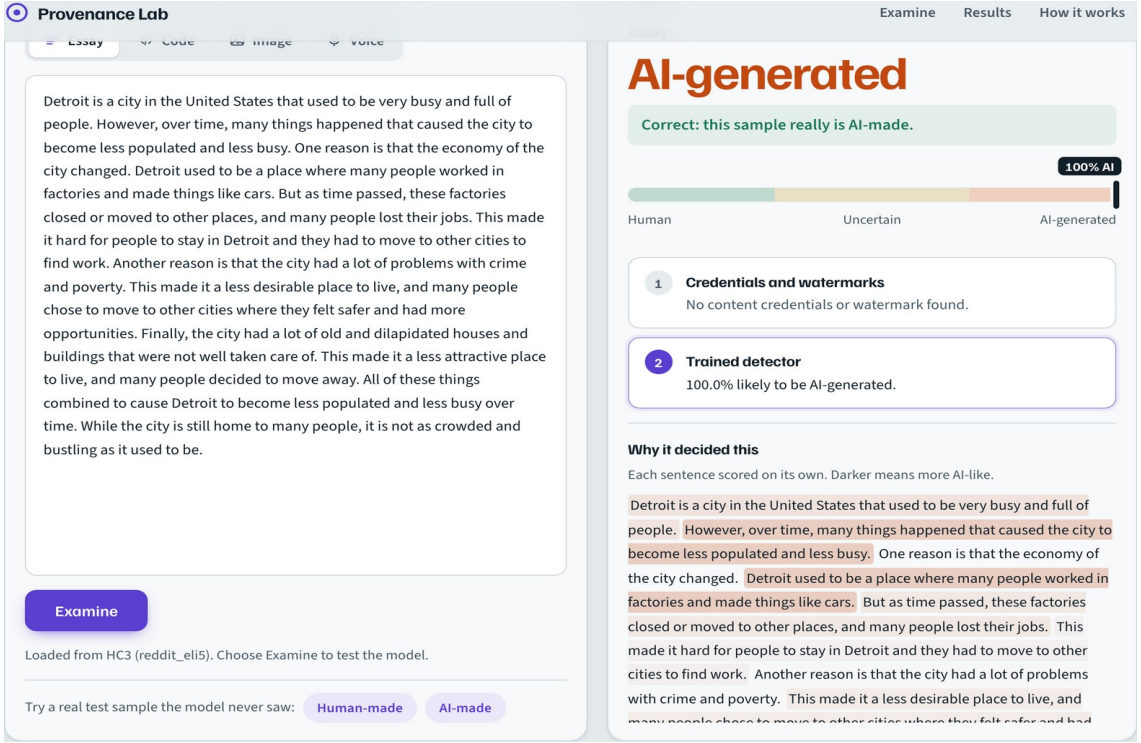
Watermarks hold only until the content is edited





C2PA image credentials were lost after a single JPEG re-save. False positives on unwatermarked human content: 0% in every test.

The working system: drop in anything, get a verdict



Web app (FastAPI + custom front end)

- Auto-detects the kind of file dropped in
- Shows which layer decided, and the AI probability on a human-to-AI scale
- Loads real held-out test samples and marks whether the model was right
- Live results page with ROC and learning curves

What we found

High accuracy on held-out benchmark data

Essays 98.3%, Code 97.6%, Images 90.7%, Voice 95.0%. Pretrained wav2vec 2.0 features lifted voice from 80.7%.

Ahead of popular public detectors

Best in 2 of 3 benchmark comparisons; a CIFAKE-specialised model trained on 16× more data edges us on images.

Benchmarks overstate real-world accuracy

All essay detectors drop on unseen models; ours fell to 57.1%, diverse training lifted it to 85.3%.

Contrastive loss: no measurable gain at this scale

Every with/without difference sits inside the 95% confidence interval, including on an unseen generator.

Proof beats prediction when it exists

Watermarks and C2PA are decisive when present but break under simple edits, so both layers are needed.

Limitations and next steps

Limitations

- Accuracy drops on unseen generators and topics (see MAGE test)
- Small training subsets on one laptop
- Unseen generators stay hard (Midjourney held out: 56% caught)
- Google's SynthID image/audio detectors aren't public; we use open equivalents
- Text and code detectors see only English and Python

Next steps

- Full datasets and longer training on a GPU
- Test voice on newer synthesis (ASVspoof 5) and fine-tune wav2vec 2.0
- Train on many more generators; retest contrastive loss with larger batches
- Extend code detection beyond Python
- Robustness training against paraphrase and compression

Thank you

Questions, and a live demo